Number to Words Converter - Approach

Saad Farooq | Technology One Engineering Exercise

This is a quick rundown of how I approached the exercise and why I made the choices I did.

How I built it

I used Test-Driven Development (write a failing test, make it pass, then move on). It's how I work most naturally, and it suited this problem well since it breaks down into lots of small cases (zero, teens, hundreds, thousands, cents, etc). It forced me to decide tricky edge cases like "what does .5 mean" up front, and the tests double as the test plan.

Project setup

ASP.NET Core Web API (C#, as preferred in the brief) for the conversion logic, a React front-end for the UI, and a separate xUnit test project. The conversion logic sits in its own class behind an interface, not in the controller, so it's testable without a web server. I considered ASP.NET's combined React+API template but kept two separate projects instead - closer to a real product and simpler to reason about.

Handling the input

The amount is kept as a plain string and split manually on the decimal point, rather than parsed into a decimal/double, since floating point can introduce rounding errors on values like 123.45. Rules I settled on: no decimal part means 0 cents; a single decimal digit like ".5" is treated as 50 cents (a tenths value, not a digit count); anything beyond two decimal digits is truncated rather than rounded, to avoid rounding carrying into the dollar amount. Reasonable defaults, not the only valid interpretation.

The algorithm itself

Built from scratch, no libraries. Two lookup arrays cover the irregular words (0-19, and the tens: twenty, thirty, etc). A helper turns any 2-digit number into words, another builds on that for 3-digit numbers (adding "hundred"/"and" where needed), and the dollar amount is chunked into groups of three digits so a scale word like "thousand" or "million" can be added per group. Zero groups are skipped so "1000" doesn't come out as "one thousand zero hundred". Cents reuse the 2-digit helper since they're always under 100.

I considered a giant lookup table mapping every number straight to its words, but that doesn't scale or show any real algorithmic thinking, and the brief asked for an original approach rather than pre-built data. I also weighed recursion vs a loop for the chunking step - recursion reads nicely for "peel off the lowest chunk and repeat", but a plain while loop does the same job without a base case or call stack to worry about, and there are only ever a handful of chunks (up to billions), so I went with the loop.

Problems I ran into along the way

Writing tests first caught a couple of bugs early. For an exact multiple of ten like 30, my first version of the 2-digit helper always appended the ones word, giving "THIRTY-ZERO" instead of "THIRTY" - fixed by checking if the ones digit is zero and returning just the tens word in that case. Same issue with hundreds: an exact hundred like 100 came out as "ONE HUNDRED AND" with nothing after it, so "AND" is now only added when there's an actual remainder. I ran into a few other issues along the way too - happy to talk through those in more detail if useful.